National University of Lesotho

Department of Mathematics and Computer Science

CS4433 Software Engineering

Individual Final Project

Moderated Learning Management System

Configuration Management and Quality Assurance

Student Name: MAKHOTSO RAMOCHELA

Student Number: 202321185

1. Introduction

This document contains a repository link, CI evidence, a static analysis report, metrics and a code review record. Table 1.1 shows where each one is.

Table 1.1: The deliverables of Phase 6

Deliverable	Where it is
Repository link	Section 2.1
Configuration management (branching, commits, tags)	Sections 2.2 to 2.4
CI pipeline and CI evidence	Sections 2.5 and 2.6
Static analysis report	Section 3
Code metrics	Section 4
Code review record	Section 5

All work of this phase was done on the branch feature/phase6-quality and merged into main. The results were measured on the source code of the Git repository, on Ubuntu Linux 24.04 with g++ 13.3, clang-tidy 18.1.3, cppcheck 2.13.0 and lizard 1.24.3.

2. Configuration management

2.1 Repository

The source code is kept in a Git repository. The link is:
https://github.com/makhotsoramochela/Software-Engineering-Project.git

The repository holds the source code, the tests, the CMake files, the CI file, the build guide (README.md) and the documents in the docs folder.

2.2 Branching strategy

The full rules are in docs/BRANCHING_STRATEGY.md. In short:

Table 2.1: Branches

Branch	Use
main	Always builds and passes all tests. Only finished work is merged into it. Nobody works directly on it.
feature/<name>	One branch for each piece of work, made from main (for example feature/phase6-quality).
fix/<name>	One branch for each bug fix, made from main (used from the maintenance phase).

A feature branch is merged into main with a merge commit (git merge --no-ff), so that the history shows where the work started and ended. On GitHub this is done with a pull request. The rule before a merge is: no build warnings, all tests pass and the static analysis finds nothing.

2.3 Commits

Each commit does one job and its first line says what was changed. Table 2.2 shows the history at the end of this phase (git log --graph).

Table 2.2: Git history

Commit	Message	Phase
5a0b3d6	Implement the Moderated LMS: business, data and presentation layers	4
6ee9d75	Add build guide, coding standard, design fidelity notes and traceability matrix	4
b86d6b7	Record the measured test coverage of the business layer (85.8%) and how to measure it	4
ed7a3d4	Simplify the code: no dynamic_cast, no callback, shorter functions	4 (tag v0.4.0)
e04e0eb	GoogleTest, system tests, coverage script, CI, three fixes	5 (tag v0.5.0)
e653647	Fix static analysis findings and split long functions	6, on the feature branch
339c7d4	Add static analysis script, local CI runner and branching strategy	6, on the feature branch
4348a47	Add the log of the local CI run and the Phase 6 change to the design differences	6, on the feature branch
b78b39d	Merge feature/phase6-quality: static analysis, metrics, code review, CI steps	6 (tag v0.6.0)

2.4 Tagged releases

Releases are annotated tags in the form vMAJOR.MINOR.PATCH. The tags v0.4.0 and v0.5.0 were added in this phase on the commits of Phase 4 and Phase 5, so that every phase can be found again. Version 1.0 is made in Phase 7 and version 1.1 in Phase 8.

Table 2.3: Tags

Tag	Commit	Meaning
v0.4.0	ed7a3d4	Phase 4: implementation
v0.5.0	e04e0eb	Phase 5: verification and validation (tests and CI)
v0.6.0	b78b39d	Phase 6: configuration management and quality assurance
v1.0	Phase 7	First release (Gate 3)
v1.1	Phase 8	Change request CR1

2.5 The CI pipeline

The file .github/workflows/ci.yml is a GitHub Actions pipeline. It runs on every push and every pull request, on a clean Ubuntu machine. Table 2.4 shows the steps. The last step was added in this phase.

Table 2.4: Steps of the CI pipeline

No.	Step	What it does
1	Get the code	Checks out the repository
2	Install the tools	Installs the compiler, CMake, SQLite, libsodium, GoogleTest, clang-tidy, clang-format, cppcheck and lizard
3	Build	cmake and cmake --build (the build must finish without warnings)
4	Unit and integration tests	ctest runs the 36 GoogleTest tests
5	System tests	python3 tests/system/run_system_tests.py build/lms (95 tests)
6	Scripted session and wrong-input test	tests/e2e/run_demo_session.sh and tests/e2e/fuzz_menus.py

No.	Step	What it does
7	Coverage	sh tests/coverage.sh (fails if the business layer has less than 70% coverage)
8	Static analysis	sh tests/static_analysis.sh (clang-tidy, clang-format, cppcheck and the complexity check)

2.6 CI evidence

Important: the pipeline has not run on GitHub yet, because the repository has not been pushed from this computer. So I cannot show a green GitHub Actions run now. As evidence I ran exactly the same steps on this computer with the script tests/run_ci_locally.sh. It builds in a new folder, runs every step in Table 2.4 and writes the file docs/CI_LOCAL_LOG.txt. Table 2.5 shows the result for commit 339c7d4, run on 30 October 2026.

Table 2.5: Local run of the CI steps

Step	Result	Status
Build	No warnings, no errors	Passed
Unit and integration tests	36 of 36 tests passed	Passed
System tests	95 tests, 95 passed, 0 failed	Passed
Scripted session	All checks passed	Passed
Wrong-input test	60 sessions, 0 problems	Passed
Coverage	85.74% of the business layer (824 of 961 lines); the limit is 70%	Passed
Static analysis	No problems	Passed

3. Static analysis report

3.1 Tools

Table 3.1: Static analysis tools

Tool	Version	What it checks
clang-tidy	18.1.3	Likely bugs (bugprone, clang-analyzer), performance, C-style casts, uninitialised variables, missing override. The checks are listed in the file .clang-tidy.
cppcheck	2.13.0	Run with --enable=all --inconclusive on the folder src: errors, style, performance and unused code.
clang-format	18.1.3	The layout of the code (file .clang-format).
lizard	1.24.3	Lines of code and cyclomatic complexity (Section 4).

All four tools are started by one script, sh tests/static_analysis.sh, which is also the last step of the CI pipeline. The script stops with an error when a tool finds a problem.

3.2 Findings

clang-tidy found **0** problems in src, before and after this phase. cppcheck found **24** points in the first run. None was an error or a warning; they were style and performance hints. Table 3.2 shows each kind and what I did.

Table 3.2: Findings of cppcheck

Finding	Count	Meaning	Action
redundantCopyLocalConst	6	A local variable copies a value that could be a const reference.	Fixed 5 (const reference). 1 kept as a copy on purpose (Section 3.3).
functionStatic	4	A member function does not use the object.	Fixed 2 (private helpers made static). 2 kept: InputReader::readFileBytes and writeFileBytes, because InputReader is one class that a test can replace.
constParameterReference	1	A parameter (Clock&) can be const.	Fixed (const Clock&).
useStlAlgorithm	8	A loop could be written with std::any_of and similar.	Kept. The loops are shorter to read for a student than a call with a lambda. They are in cppcheck-suppressions.txt.
unusedFunction	4	isLocked, fromMinutes, minutes and isSeen are never called in src.	Kept. They are operations of the design (class diagram), and the check looks at src only. They are in cppcheck-suppressions.txt.
useInitializationList	1	A member is set in the constructor body (Windows-only code in InputReader.cpp).	Kept. The member exists only on Windows.

Result: 8 fixed, 16 kept with a reason (15 in cppcheck-suppressions.txt and 1 inline). A kept finding is written in the file cppcheck-suppressions.txt with its reason, so that the script passes and the reason is not lost. After the fixes, clang-tidy, clang-format and cppcheck all report nothing new.

3.3 A fix that would have been a bug

cppcheck asked for a const reference for the local variable oldFinal in ModerationService::adjust. I changed it, and the test Workflow.moderation_approve_adjust_return_and_release failed: the audit entry showed the new mark as the old mark. The reason is that a reference points into the submission, and the submission is changed a few lines later, so the "old" mark changed too. I put the copy back, added a comment that says why the copy is needed, and marked the line as a kept finding. This shows why every suggestion of a tool must be checked with the tests.

4. Code metrics

The metrics were measured with lizard on the folder src (the program) and the folder tests (the C++ tests). NLOC means lines of code without blank lines and comments. CCN is the cyclomatic complexity of a function (1 plus the number of decisions).

4.1 Size

Table 4.1: Size of the code at the end of Phase 6

Part	NLOC	Functions	Average NLOC per function	Average CCN
Business layer (src/business)	2,012	220	5.7	2.1
Data layer (src/data)	873	95	6.7	1.7
User interface (src/ui)	1,154	80	11.1	3.2
main.cpp	21	-	-	-
Program in total	**4,060**	**396**	**7.0**	**2.2**
C++ tests (tests)	801	48	15.3	2.0

The tests are 801 lines for 4,060 lines of program (about 1 line of test for every 5 lines of program), not counting the Python system tests. The program has 63 source and header files and about 180 lines of comment.

4.2 Complexity

I set a limit: **no function with a CCN over 10**. A menu function that is only a switch with one short call for each choice is allowed to be over, because each case is simple and the function is easy to read.

At the start of this phase 7 functions were over the limit. Five were split into smaller functions, and 2 are menu switches that I kept. Table 4.2 shows them.

Table 4.2: Functions that were over the limit

Function	Before	After	What was done
DateTime::parse	16	4	Helpers readParts, isRealDate and isRealTime
AssignmentService::grantExtension	12	10	The recalculation of late work moved to recalculateLateWork (BR-04)
Application::run	12	10	First-start code moved to createFirstAccounts; two identical catch blocks joined
SubmissionService::prepareSubmission	11	9	The refusal texts moved to refuseReplacing
InputReader::readDecimal	11	3	The number check moved to parseDecimal
AdministratorMenu::handle	18	18	Kept: a switch with 16 short cases
HeadOfDepartmentMenu::handle	11	11	Kept: a switch with short cases

The two kept functions are in lizard-whitelist.txt. After the change no other function is over 10, and the average CCN is 2.2 (it was 2.3).

4.3 Key functions

Table 4.3: Complexity of the key functions

Function	NLOC	CCN	Rule
AuthService::login	24	10	FR-01, FR-03 (lock)
SubmissionService::prepareSubmission	39	9	FR-16 to FR-18, BR-03
SubmissionService::confirmSubmission	34	7	FR-17, BR-03
MarkingService::saveMark	24	6	FR-12, BR-02, BR-06
Menu::run (Template Method)	17	5	AR-6
ModerationService::adjust	20	4	BR-07, BR-08
Database::ensureSchema	22	4	AR-3, DEF-05
TenPercentPerDayPenalty::apply	8	3	BR-02 (Strategy)

The functions that hold the business rules are short and simple. AuthService::login is at the limit (10), because it has the checks for an unknown user, an inactive user, a locked account and a wrong password. It is already split (handleWrongPassword and saveUser).

5. Code review record

I reviewed my own code against the checklist in Table 5.1. For each item I used a search of the code (grep) or read the code, and I wrote down the result. A review of your own work can miss things, so the items that can be checked by a tool were checked by a tool.

Table 5.1: Checklist, result and change

No.	Check	Result	Change
1	Layers (AR-1, AR-2): no SQL or libsodium in the business layer or the menus	No include of sqlite3.h or sodium.h outside src/data and the platform files. The only match was a comment.	None
2	SQL is parameterised (AR-3)	Every value is bound. The only text added to a SQL string is the constant list of column names.	None
3	Permission check first in each service method (AR-4)	16 of 16 methods of AdminService check first. Methods of other services check first or call a helper that checks (for example requireSubmissionOfDepartment). exportFile checks the role itself with can(), for two roles.	None. Noted: exportFile does not use requirePermission.
4	Time only through Clock (AR-7)	std::time is used only in SystemClock.	None
5	No plain password is shown or logged	No output statement prints a password; the screen only says that it was saved.	None
6	No new, delete, malloc or free	None in src. Smart pointers and objects are used.	None
7	No C-style casts and no dynamic_cast	None. (The dynamic_casts were removed in Phase 4.)	None
8	No TODO or FIXME left	None in src and tests.	None
9	No magic numbers	The number 1000000 (the biggest id) was typed 19 times in the menus.	**Changed:** InputReader::readId with the named constant MAX_ID.
10	Function length and complexity	7 functions over CCN 10 (Section 4.2).	**Changed:** 5 functions split; 2 kept with a reason.
11	Duplicate code	Two catch blocks in Application::run were the same.	**Changed:** joined into one.
12	Names and comments say the requirement or rule	Rules are marked in the code (BR-04, FR-13, AR-8 and so on). New helpers have a comment.	Comments added to the new helpers.
13	Layout and warnings	clang-format and the compiler report nothing.	None
14	Tool suggestions are safe	A suggestion of cppcheck changed a mark in the audit entry (Section 3.3).	**Changed back** and commented.

5.1 What changed because of the review

- The repeated number 1000000 now has a name and one place (InputReader::readId and MAX_ID).
- Five long functions were split, and the two switch functions that stay over the limit are listed with the reason.
- The repeated catch block in Application::run was removed.
- The 18th difference between the code and the design (new small helpers) was written in docs/DESIGN_FIDELITY.md, so that the artefacts still describe the same system.
- After all, changes the tests were run again: 36 of 36, 95 of 95 system tests, 60 wrong-input sessions with 0 problems, and coverage 85.74%.

.